

Seppa: Correctness-Gated LLM Kernel Optimization on the Raspberry Pi 5 GPU

Adam Munawar Rahman

New York University

New York, NY, USA

mrs541@nyu.edu

ECE-GY 9953 Advanced Project. Advisor: Prof. Brandon Reagen

Abstract—Most edge AI deployments on single-board computers like the Raspberry Pi leave the integrated GPU idle. We present Seppa, a harness that puts a large language model in the loop for optimizing GPU kernels. The model proposes changes, then a finite-state machine compiles, verifies against physics-based correctness gates, benchmarks, decides keep or revert, and refuses to benchmark any kernel that failed verification. The target is the flood-simulation stencil of an offline flood-guidance application, a kernel in which each grid cell updates from its neighbors. On that stencil, a hand-driven sweep through the same gates produced a kernel 1.58x faster than the original. The machine re-measured and re-judged the result in five consecutive runs driven from a second computer, and in every run it refused to benchmark a kernel that failed its gate. With concurrent CPU inference, that 1.58x advantage widens to 2.20x. The deployment sustains 9.6 tokens/s of language-model decode alongside 883 simulation steps per second, which beats the strongest CPU-only arrangement measured (simulation and model on separate cores) on both axes in the steady-state campaign.

Index Terms—GPU kernel optimization, LLM agents, correctness verification, edge computing, Vulkan

I. INTRODUCTION

Raspberry Pi class boards are a common platform for edge AI in emergency-response and field settings, where budget, connectivity, and power are constrained [12] and inference must happen on-device in network outages [11]. A Pi 5 can already run a quantized billion-parameter model on its CPU cores [8]. AI capability is usually added with dedicated hardware, such as a neural-processing add-on board (NPU HAT) or a USB accelerator [13], which raises cost and adds a procurement dependency in a supply-constrained chip market [14].

Less attention is given to silicon already on the board: the Pi 5 ships a VideoCore VII GPU (also referred to as the V3D) that compute workloads scarcely touch, and it idles during CPU inference. We explore two questions regarding the worth of the idle GPU. The first is whether it can be made fast enough at a useful workload to serve as a co-processor. The second is whether it can run alongside a busy CPU on a shared LPDDR memory bus. The case study (Section V) and its machine-checked reproduction (Section VI) answer the first; the concurrency metrics (Section VII) answer the second.

The V3D is a mobile GPU with both limited power and limited documentation. Tuning habits learned on NVIDIA GPUs, the target of most optimization literature, largely fail

(Section II provides the background). Work of this kind is what the field has begun handing off to large language models (LLMs). Most existing tools are agentic AI optimizers such as AutoKernel [15], which are loops that place the model in charge of every step, including checking its own results. These optimizers guide the model with prompts and trust it to verify honestly and revert bad kernels, and Section III’s related work shows that trust has a poor record. Seppa removes the need for that trust by narrowing the model’s role: the model proposes, the server executes. A finite-state machine built on Apache Burr [5] (a Python state-machine library) owns compilation, verification, benchmarking, and the verdict. It refuses out-of-order requests. The machine is served over the Model Context Protocol (MCP) [6], the standard interface through which a language model calls tools on a server.

The representative workload is a flood simulation for an offline flood-guidance application [7], which must run beside LLM narration on the same board. Its core kernel is a stencil, a computation that updates every cell of a grid from its neighbors each step. That kernel is the optimization target, and the application’s CPU-GPU split is what we measure (Section IX describes the application).

This paper shows that a machine can verify, reproduce, and gate the model’s results. It does not show that the model can find good optimizations on its own. Within that scope, this paper reports:

1. the optimization of a real flood stencil for the Pi’s V3D GPU to 1.58x under three physics gates, failed hypotheses included (Section V);
2. an action-space lesson: the first search plateaued because every winning change lived in the host program (the CPU-side code that allocates buffers and launches the kernel), outside what the machine let the model edit (Section V-C);
3. a machine-checked reproduction over MCP, repeated five times under defined pass criteria (Section VI);
4. an interference measurement of the deployed configuration against a gate-verified CPU-only counterfactual (Section VII). In the steady-state campaign the CPU-GPU split beats the strongest CPU-only arrangement on both axes; that arrangement gives the simulation and the model separate cores.

TABLE I
PLATFORM, COLLECTED FROM THE RUNNING BOARD BY
COLLECT_SPECS.SH

Model	Raspberry Pi 5 Model B Rev 1.1
SoC	BCM2712
CPU	4 x Cortex-A76 @ 2400 MHz
RAM	16 GB LPDDR (shared with GPU)
OS / kernel	Debian GNU/Linux 13 (trixie) (DietPi 10.5.2), 6.18.39+rpt-rpi-2712
Firmware	2025/12/08

TABLE II
GPU COMPUTE LIMITS, COLLECTED LIVE (VULKANINFO)

Device	V3D 7.1.10.2
Driver	Mesa 25.0.7-2+rpt4 (v3dv)
Vulkan API	1.3.305
Core clock	960 MHz
Max invocations / workgroup	256
Shared memory / workgroup	16 KB
Subgroup (SIMD) width	16
fp16 arithmetic	no
Cooperative matrix	no

II. BACKGROUND: THE RASPBERRY PI 5 GPU

The Pi 5 pairs four Cortex-A76 CPU cores with a VideoCore VII GPU (V3D 7.1), the block that also drives the desktop. There is no CUDA and no vendor compute toolchain; compute reaches the GPU only through Vulkan shaders (small programs the GPU runs many times in parallel), compiled by Mesa’s v3dv driver [9]. In this paper, kernel refers to a GLSL compute shader.

Tables I and II list the platform and the GPU’s compute limits, collected from the running board with `pi/collect_specs.sh` (archived with the raw `vulkaninfo` dump). The limits are tight, and the scale is modest: the best kernel measured sustains about 13 GFLOP/s in fp32, and that figure belongs to a dense matrix multiply (Section VII-A compares the processors on the stencil itself). The GPU is a second, slower, but “free” engine.

In Vulkan’s terms, a dispatch is one launch of a kernel over the whole grid, and an invocation is one execution of the kernel by one GPU thread. Invocations are grouped into workgroups, and the hardware runs invocations in lockstep bundles of 16 called subgroups.

The driver exhibits some unusual behavior: when a shader wants more registers than actually exist, v3dv does not fail. It walks a ladder of thirteen compile strategies [16], disabling, one at a time, the optimizations that spend registers for speed: instruction scheduling, code motion, loop unrolling, load sorting, pipelining. It then halves the thread count, repeats the disables, and ends on a fallback scheduler, handing back whatever survives, sometimes several times slower.

III. RELATED WORK

Kernel generation is now a common LLM task. KernelBench [3], the standard benchmark, only credits a speedup

when the kernel also passes its correctness check. Kevin [17] trains a model with reinforcement learning, and TritonRL [18] trains a model to write kernels in Triton (a Python-like kernel language) “without cheating.” In spite of this, CUDA-L1 [4] reports that 82 of its 250 RL-generated kernels (32.8%) exploited stream-timing loopholes, arranging the GPU’s work queues so the timed section ended before the real work did, and inflating the reported speedup to 18x. Containing it required a reward checker, a database of known hacks, and forced stream synchronization. When the reward is a measured value, the model learns to optimize the measurement itself.

A passing check does not indicate a correct kernel. Sarkar [19] tests the oracles these benchmarks use: the automated checks that decide whether a kernel counts as correct. The study finds them systematically optimistic, with seeded-buggy kernels passing the benchmark oracle while failing a stricter one. SpecBench [20] states generally: when oversight collapses onto an automated suite, the agent optimizes the suite. Our physics gates inherit the same weakness, which is noted in the limitations in Section VIII.

Another option is to put the evaluator outside the model instead of arguing with it. AlphaEvolve [22] and FunSearch [21] use the model as a mutator inside a loop scored by a fixed evaluator. AlphaEvolve found a 48-multiplication procedure for 4x4 complex matrix multiplication, the first improvement in that setting in 56 years. Seppa makes the same structural commitment, keeping the scorer outside the model, with the scoring expressed as reachability in a graph instead of a fitness function (Section IV). In LLM-aided hardware design the same split appears with a human in it: Chip-Chat [10] carried conversational design to tapeout with an engineer and a testbench closing the loop, and VeriGen [23] fine-tunes open models for Verilog. We close the loop with a state machine instead.

IV. THE SEPPA HARNESS

Seppa (Finnish *seppä*, smith) is a port of AutoKernel [15], an open-source kernel-optimization loop, onto a finite-state machine built with Apache Burr. Theodosia, an adapter that mounts a Burr state machine as an MCP server, serves it from the Pi itself.

Fig. 1 shows the graph. Two guard edges lead to `log_variant`, one on compile failure and one on gate failure, and a third edge, omitted from the figure, runs from implement back to hypothesize when the model submits nothing usable. The guards are ordinary code, from `v3d_flood2_opt.py`:

```

1 ("compile_", "verify", expr("compile_ok")),
2 ("compile_", "log_variant", expr("not compile_ok")),
3 ("verify", "benchmark", expr("verify_ok")),
4 ("verify", "log_variant", expr("not verify_ok")), # guard
5 ("benchmark", "evaluate"),
6 ("evaluate", "log_variant"),

```

The benchmark state is reachable only through a green `verify`. In operation, the server runs on the Pi as a Python program, and an MCP client drives it through one tool,

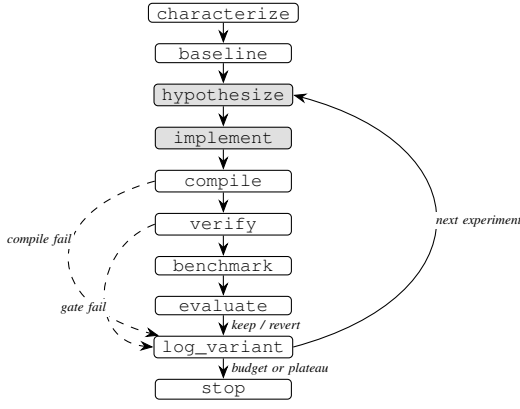


Fig. 1. The Seppa loop. Shaded states are the agent’s; dashed guard edges route compile and gate failures to log_variant, so benchmark requires a green verify.

step(action, inputs), receiving each state’s result and the legal next actions; anything else is rejected. The server also exposes session utilities (reset_session, fork_at, resource reads) that no archived session used; Section VIII notes the resampling risk fork_at would pose.

The port preserves AutoKernel’s loop discipline (audited in docs/autokernel_fidelity.md): one focused change per experiment, a measured baseline, correctness and timing taken from the same execution, keep only on a gain of more than 1%, and all experiments recorded. The difference is in the trust architecture. AutoKernel trusts its agent to run the benchmark and honestly revert failures; in Seppa the verdict is a deterministic action the model cannot reach. A run ends after three consecutive non-improvements on gate-passing variants, or at the prescribed budget.

The author built the application and the harness, chose the physics gates, ran the hand-driven sweep of Section V-B, and supervised the sessions. The language model was Anthropic Claude Sonnet 5, driven through the Claude Code client at high reasoning effort. It answered each hypothesize prompt by writing implement’s inputs, the kernel source and the host-side parameters, and nothing else. Every measurement and verdict came from the machine, and no proposed kernel was edited by hand (transcript in docs/paper/artifacts/claude_sessions/). Concretely: the winning kernel came from the hand-driven sweep, and the machine verified and reproduced it (Section VI). In the one fully agent-driven session the model wrote its own shader pair for each strip value it tried. Of the two parameters the server had already named, it worked only strip, and it never proposed the fused kernel of Section V-B.

For the flood target, verify runs the candidate kernel for 400 steps at 256x256 and applies three checks. Each enforces a different standard of correctness, the usual approach in simulation verification [24]. First, accuracy: the GPU result must match the same simulation run on the CPU in double precision, within a normalized mean-square error (NMSE) of 1e-3. The tolerance forgives 32-bit rounding drift but is

six orders of magnitude above what correct kernels measure, so real bugs land far outside it. Second, conservation: the simulation adds water only as rainfall and is built to conserve mass [1], so total water in the grid must stay within 2% of what rained in. Correct kernels drift about 0.003%. Third, plausibility: water flows downhill, so the deepest water must end up inside the terrain’s basin. A passing gate prints this in the harness log (here a 4,000-step endurance run; verify itself runs 400 steps), long lines wrapped:

```

1 selected: V3D 7.1.10.2
2 grid=256^2 steps=4000 time=1.880s 3.07 GFLOP/s
3 correct(NMSE vs CPU)=yes NMSE=1.26e-09
4 mass: rain_injected=5242879.9
5 water_total=5242739.3 (conserved vs rain=yes)
6 pooling: max depth=111.718 at (127,127)
7 basin_centre=(128,128) (pools in basin=yes)

```

V. CASE STUDY: THE FLOOD STENCIL

A. The Workload

The simulation runs on a square grid of water depths, and each time step every cell updates from its four neighbors, which is an access pattern called a stencil. The update takes two passes: a flux pass computes how much water flows across each cell boundary, and a height pass applies those flows to produce each cell’s new depth. The original implementation ran about 1,045 steps/s at 256x256 (1.51 GFLOP/s). Both shaders compile without triggering Section II’s fallback ladder, which means they never stress the register allocator. Therefore, the bottleneck was something other than register pressure.

B. Falsification Sweep

We treated the bottleneck as something to test, one hypothesis at a time. Each plausible explanation for the kernel’s speed became one shader variant, and each variant went through the same compile-verify-benchmark loop that judges the model’s proposals. The ideas themselves are simple. Packing stores two values in one memory word so the kernel issues fewer loads. Fusion merges the two passes into a single kernel launch, removing the buffer and the synchronization barrier between them. Strip-mining gives each GPU thread a short column of cells instead of one, spreading the fixed cost of starting a thread over more work. Every variant passed all three gates.

Speeds come from 400-step runs at 256x256 on the parameterized vkflood2 harness. With identical kernels, the vkflood2 host loop is faster than the original application’s, about 1,345 steps/s against 1,045. Every comparison in this paper is within vkflood2 to avoid inflating the ratio by mixing the two harnesses. Table III shows the sweep. Percentages in the table are relative to its baseline, about 1,345 steps/s. That baseline is 1.94 GFLOP/s; the packed variant’s 1.93 is within noise of it, hence no change.

The outcome ran against the obvious expectation. Halving the flux pass’s memory traffic changed nothing, and fusion bought only 5%. Fixed per-invocation cost dominated: each invocation does so little arithmetic that launch overhead overtakes it, and giving each thread two cells beat every memory-system optimization attempted; four cells gave the gain back

TABLE III
FALSIFICATION SWEEP: 400 STEPS AT 256x256 IN THE VKFLOOD2
HARNESS

Variant	Hypothesis tested	Result
Packed vec2 state (water, surface) halving flux-pass loads	Memory-op count is the bound	1.93 GFLOP/s, no change: falsified
Fused single-dispatch step (flux buffer eliminated)	Traffic and barriers are the bound	2.04 GFLOP/s, +5%: mostly falsified
2 cells per invocation (strip-mined dispatch)	Fixed per-invocation cost is the bound	2.40 GFLOP/s, +23%: supported
4 cells per invocation	More strip is better	2.35 GFLOP/s: falsified, register pressure
Fused + 2 cells per invocation (fused2s.comp)	The wins stack	3.08 GFLOP/s, +59%

to register pressure. The sweep’s own numbers size that cost: strip-2 removes 65,536 invocations per step and saves about 140 microseconds, near 2 ns per invocation. That is about two clock cycles, the scale of starting a thread itself.

Two hardware details shaped the final kernel. First, the strips run vertically. A subgroup’s 16 lanes (its 16 invocations) are horizontal neighbors. When each lane walks down its own short column, the subgroup’s loads at every row stay on adjacent addresses, the pattern the memory system serves most efficiently. Second, the kernel consumes each flux value as it is produced, because holding all four alive fails register allocation. The shipped kernel is 1.58x at 256x256: 1.574 to 1.585 across the five scored reproductions, 1.571 in the July transcript quoted in Section VI, and 1.59x hand-timed in the sweep. It is 1.41x at 512x512 and 1.18x at 1024x1024, and it holds all gates over 4,000 steps.

C. Action-Space Limits

An earlier version of the harness permitted the model to edit only the shader source, and a session run against this stencil under that restriction found nothing. We nearly concluded that the kernel had hit its limit. The restriction itself was the problem: every winning change in Table III lives outside the shader. Packing modifies the buffer layout, fusion removes a host-loop stage, and strip-mining changes the dispatch shape. All three are host-program decisions. Once `implement` accepted the host parameters as well (`{shader, height_shader, strip}`), the machine verified and kept the fused strip-2 kernel, starting from its own baseline (Section VI). A plateau can mean the search space is too small even when the hardware still has headroom.

Two other targets ran through the same gated loop, each behind its own correctness gate. A GEMM (dense matrix multiply) improved to 13.42 GFLOP/s over two rounds, and in a `llama.cpp` matrix-vector kernel, replacing the unrolled inner loop with a plain loop gained 28% in text-generation speed. The running notes record the matrix-vector result and the GEMM’s final round, 12.56 to 13.42 GFLOP/s, and the shaders ship as `pi/gemm.comp` and `pi/gemm-best.comp`.

VI. MACHINE-CHECKED REPRODUCTION OVER MCP

The Pi runs the Theodosia server, which mounts the Seppa state machine built on Apache Burr. A laptop runs the driver script `drive_flood2_mcp.py`, which connects over MCP and issues the loop’s actions in order through the `step` tool. All measurement happens on the Pi, and the driver just issues requests.

The state machine measures its own baseline (1,346.8 steps/s on 2026-07-11, gates green), receives the fused strip-2 kernel as experiment 1, compiles it, gates it, benchmarks 2,116.4 steps/s, and issues its own verdict. Numeric fields are rounded here to one decimal:

```
1 {"exp": 1, "fused": true, "strip": 2,
2  "compile_ok": true, "verify_ok": true,
3  "steps_per_sec": 2116.4, "best_sps": 2116.4,
4  "verdict": "keep"}
```

Then the driver tries to cheat: it submits the same kernel with rainfall injection doubled in one of the two cell updates, a change that compiles and breaks conservation. The gate fails it; the driver requests `benchmark` anyway, and the server answers (two advisory fields elided):

```
1 {"error": "invalid_transition",
2  "requested": "benchmark",
3  "valid_next_actions": ["log_variant"],
4  "message": "action 'benchmark' is not
5   reachable from current state. Valid
6   actions now: ['log_variant']."}

```

The variant is recorded as a revert with a null `steps_per_sec`. Gates and timing come from one execution, so the caller still sees the failing kernel’s speed (`verify` reports it as `steps_per_sec_if_kept`); the green gate controls the ledger, the verdict, and the kept kernel.

To show repeatability, `passk_flood2.py` runs the two-cycle reproduction `k` times from cool starts, each run held until the board drops below 55 °C. A run passes when the baseline clears its gates inside 1,300 to 1,400 steps/s, the fused kernel is kept at 1.5x or better, and the broken kernel is refused and nulled. Five of five runs passed on 2026-08-09: baselines $1,348.6 \pm 4.1$ steps/s, the kept kernel 2,127.7 in every run (identical at the timer’s resolution), speedups 1.574 to 1.585.

One session was fully agent-driven (2026-08-02, a three-experiment budget given to the proposer). The server’s `characterize` payload names fixed per-invocation cost as the bound and exposes both host-contract parameters, fusion and strip. The `hypothesize` prompt states on every turn that an empty `height_shader` requests a fused single-dispatch step. The model began with the bound named and both parameters exposed, so the session demonstrates the loop’s enforcement alone. The model worked only the strip parameter, writing a fresh strip-mined shader pair for each value: strip 4 kept at 1,470.6, +8.8%; strip 8 hit the register cliff and was reverted at 921.7; strip 6 probed it and was reverted at 1,298.7. It never reached fusion, and all of its reported numbers came from the machine.

VII. CONCURRENT CPU AND GPU EXECUTION

In deployment the GPU drives the simulation while the CPU computes shelter routes and runs the language model (Granite 4.0 1B on llama.cpp with `-ngl 0`, no layers offloaded to the GPU; 4 threads). Decode, the token-by-token generation phase, is memory-bound: its speed is set by how fast model weights stream from memory, which matters because the two processors share one LPDDR bus.

The campaign first surfaced one configuration surprise. With any Vulkan device visible, llama.cpp places CPU-resident model weights in memory set aside for GPU transfers (host-pinned and write-combined) even at `-ngl 0`; that memory is slow for the CPU reads that dominate decode. Hiding the device with `GGML_VK_VISIBLE_DEVICES=99` increases decode from 9.68 to 11.59 tokens/s, a 20% gain from an environment variable (A/B from cool starts, llama-bench, 5 repetitions; raw logs under `ab_vkvisible` in `docs/paper/artifacts/`). Every number below uses the hidden-device configuration that the application ships.

The benchmark script `conc_steady_bench.sh`, under `pi/flood/`, measures three conditions: each kernel alone, decode alone, and each kernel looping while decode runs. Each decode phase gets a cooldown, then a soak of eight decode repetitions that brings the chip to working temperature. The soak’s own repetitions stay out of the tables. Kernel-alone phases run straight from the cooldown, so they finish before heat accumulates.

Each decode phase defines the measurement window. The llama-bench tool generates 64 tokens per repetition for 20 repetitions, long enough for 12 to 28 flood completions, and a flood run counts only if it fits inside that window.

The Pi’s firmware defends against heat by lowering clocks, gently past a soft temperature limit and sharply at a hard throttle, and the logs record which was active. The soak runs a fixed count instead of waiting on those flags, so the phases landed in different regimes. Inside the measurement windows the concurrent phases ran soft-limited 85 to 88% of the time and carried the hard-throttle flag for another 10 to 13%, peaking at 76.3 °C. Decode alone reached 73.0 °C and was limited for 3%.

An earlier short-window campaign, preserved in `conc_bench2/`, read 711.9 steps/s concurrent with decode at 10.3, against this run’s 883.4 and 9.6. Aligning the two campaigns on time since decode began resolves the difference. Concurrent throughput follows a settling curve. Near 73 °C the firmware’s soft limit begins capping CPU clocks, decode slows from 10.2 to 9.6 tokens/s, its per-second memory traffic thins, and the flood rate climbs. The thermal logs record the cap engaging in both campaigns.

The earlier campaign’s window covered decode’s first half-minute, before the cap settled: 711.9 ± 76.1 over its five completions, against 663.3 ± 34.4 for this run’s completions over the same seconds. The segment’s decode rates match as well, 10.32 against 10.21, the latter from this run’s soak over those same seconds; notebook

TABLE IV
CONCURRENT GPU FLOOD AND CPU LLM DECODE

Condition	GPU flood (steps/s)	CPU decode (t/s)
Optimized kernel alone	2,128.0 $\pm$ 1.7 (n=3)	
Original kernel alone	1,351.4 $\pm$ 0.5 (n=3)	
Decode alone, 4 threads (post-soak)		11.4 $\pm$ 0.0
Concurrent, optimized kernel	883.4 $\pm$ 47.1 (n=28)	9.6 $\pm$ 0.2
Concurrent, original kernel	401.8 $\pm$ 8.3 (n=12)	9.6 $\pm$ 0.2

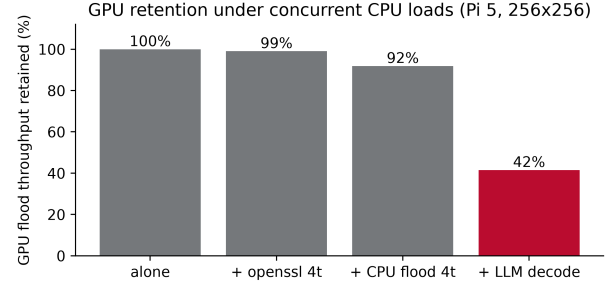


Fig. 2. GPU flood throughput retained beside three concurrent CPU loads, as a percentage of the same kernel running alone. Compute-bound and memory-bound CPU work barely disturb the GPU; language-model decode, which streams weights from DRAM every token, takes it to 42%. Raw logs under `genload2` and `conc_steady` in `docs/paper/artifacts/`.

`02_concurrency_envelope.ipynb` in the repository derives the alignment. The window reported here sits entirely in the capped regime, where the rate still climbs from 818.6 steps/s over the first quarter to 919.1 over the last. The concurrent rate below is therefore a window mean on a rising curve. Results from the steady-state run of 2026-08-10 are in Table IV (raw logs under `conc_steady` in `docs/paper/artifacts/`).

Three things follow. Co-processing costs the CPU 16% of its decode and buys a continuous 883 steps/s of simulation that a CPU-only deployment does not have. The interference is lopsided: the CPU keeps 84% of its rate while the GPU keeps 42%, as decode traffic crowds the shared bus. And contention favors the optimized kernel: 1.58x over the original alone becomes 2.20x concurrent, at identical decode cost (9.6 t/s under either kernel). Because the rate is still climbing across the window, 883.4 is a conservative estimate of the late-window rate.

Decode is also the GPU’s worst case (Fig. 2). Paired with a compute-bound CPU load (openssl SHA-256, four threads), the GPU keeps 99% of its rate and the load keeps 88% of its solo throughput on 16 KB blocks. Paired with the memory-bound four-thread CPU flood, the GPU keeps 92% and the load keeps 75%. Both pairings used continuous flood-simulation probes over cool-start 60 s windows, and both sides are logged under `genload2` in `docs/paper/artifacts/`.

These two pairings also bound the thermal confound. Both ran at the same 76.3 °C, with the soft limit active for 80% of the openssl phases and 70% of the flood phases. A solo

TABLE V
THE CPU-ONLY COUNTERFACTUAL

Condition	Flood (steps/s)	CPU decode (t/s)
CPU flood alone, 1 / 2 / 4 threads	992.5 / 1,980.5 / 3,852.4	
Decode alone, 3 threads (pinned)		11.9 $\pm$ 0.0
Partitioned: flood 1t + decode 3t	678.8 $\pm$ 10.6 (n=25)	8.4 $\pm$ 0.1
Oversubscribed: flood 4t + decode 4t	857.1 $\pm$ 181.0 (n=89)	3.0 $\pm$ 0.3
GPU concurrent, optimized (from above)	883.4 $\pm$ 47.1	9.6 $\pm$ 0.2

phase never rose above 57.6 °C. The GPU still kept 99% and 92%, so clock governance does not explain decode’s 42% on its own. What separates the three loads is how much memory traffic each one generates: the heavier that traffic, the more the GPU loses, by 1% against openssl, 8% against the CPU flood, and 58% against decode.

A. The CPU-Only Counterfactual

Four idle A76 cores run this stencil at 3,852 steps/s, well above the V3D’s 2,128, so the GPU seems unnecessary. The CPU implementation, `cpuflood.cpp` in `pi/flood/`, is the same update written with OpenMP, and it passes the same three gates. Its error against the double-precision reference is NMSE 1.46e-11, far inside the 1e-3 tolerance (gate logs in `cpu_flood_bench/`).

The companion script `cpu_steady_bench.sh` measures it alone, then sharing the cores with decode in two arrangements: pinned apart (flood on core 0, decode on cores 1 to 3), and oversubscribed with four threads each. The same soak-and-measure protocol applied; raw logs are under `cpu_steady` in `docs/paper/artifacts/`. Table V presents the outcome.

The idle-core number is real but unavailable, because in deployment decode is always running. Oversubscribed, decode collapses 75%: llama.cpp’s workers synchronize every token, so losing one core stalls all four. That row’s flood mean is inflated by bursts between decode repetitions, hence its ± 181 . Partitioned is the best CPU-only arrangement, and decode there pays a 29% tax against the GPU split’s 16%, each measured against its own solo configuration.

A third scheme, time-slicing, would flood about 23% of the time to match 883 steps/s, cap decode at 8.8 t/s, and freeze guidance during bursts. Against the partitioned arrangement the GPU split wins on both axes: 30% on simulation (883.4 $\pm$ 47.1 against 678.8 $\pm$ 10.6) and 14% on decode (9.6 $\pm$ 0.2 against 8.4 $\pm$ 0.1). The slower processor wins because the CPU has nothing left to spare.

The size of that margin depends on the campaign; its direction does not. In the earlier campaign the decode axis holds and the simulation axis ties within noise (Section VIII), and no measured campaign puts the partitioned scheme ahead on either axis.

VIII. LIMITATIONS

The measured envelope is one board (Pi 5, V3D 7.1.10.2, Mesa v3dv 25.0.7) and one grid family, and the speedup shrinks as the grid grows. An operating-system update (Linux kernel 6.12 to 6.18) separates the July and August campaigns. Flood baselines agree across it: 1,346.8 before, 1,348.6 $\pm$ 4.1 after. The board became unresponsive after the August campaign, so none of these quantities can be re-measured until new hardware is provisioned.

The gates are necessary conditions: verification covers one storm scenario at one grid size. They caught what they were built to catch: the deliberately broken kernel of Section VI was refused in all five scored runs. The keep decision rests on a single timing sample against a 1% threshold. Across Section VI’s five runs the baseline spread was about 0.3%, and the kept kernel repeated to the timer’s millisecond resolution; at the kept kernel’s 0.188 s run time, that granularity is a 0.53% step.

The 3,852 steps/s CPU figure assumed a cache-resident 256x256 working set and may not hold at larger grids.

Section VII has three asymmetries. The GPU kernel was tuned and the CPU counterfactual was not; `cpuflood.cpp` runs the original two-pass scheme. The thermal operating points were not matched, so clock governance accounts for part of the 16% decode cost. And the settled regime was measured once: the earlier campaign in `conc_bench2/` sampled the pre-cap transient (Section VII aligns the two), so no second measurement of the settled operating point exists.

The partitioned counterfactual does replicate: 678.4 $\pm$ 63.3, 680.6 $\pm$ 22.9, and 678.8 $\pm$ 10.6 steps/s across the three campaigns that measured it, with decode at 8.38 and 8.41 in the two that used the shipped hidden-device configuration. Taken at face value, the earlier campaign still narrows the GPU split’s simulation margin over the partitioned scheme to within noise: 711.9 $\pm$ 76.1 against 678.8 $\pm$ 10.6. The decode margin holds in both campaigns (9.6 and 10.3 against 8.4), and no measured campaign puts the partitioned scheme ahead on either axis.

The loop’s ability to discover optimizations on its own is untested here. The winning kernel came from the hand-driven sweep, and the agent-driven session worked only the strip parameter of the two the server itself named. The server’s `fork_at` utility rewinds to an earlier state, so a caller could resample a marginal kernel until one sample cleared the 1% threshold, and `evaluate` would not notice. Theodosia exposes `fork_at` on every mounted server, so this harness cannot switch it off; no archived session used it.

Full GPU offload of the language model is blocked by an upstream llama.cpp Vulkan defect. Some numbers have no archived log: the original application’s 1,045 steps/s, the sweep’s GFLOP/s column and hand-timed ratios, the 1.41x and 1.18x at larger grids, and the GEMM and matrix-vector results. These come from the repository’s dated running notes and shipped kernel files. Recreation by the author is pending replacement hardware.

IX. THE SAMPLE APPLICATION

The sample application [7], designed to function offline, simulates surface flooding over real terrain [1], finds shelter routes by mobility profile [2], and explains the result in plain language. The model only narrates what the deterministic code computed and does not make a safety decision. The application lives at github.com/msradam/bonbibib and was built as an Arm AI Optimization Challenge entry for August 2026; the harness discussed in this paper is located at github.com/msradam/seppa. The harness, the gates, the optimized kernel, and every measurement reported here are new work for this course, built on top of that application. Measurements used the Granite 4.0 1B configuration of the representative application. The model file (a 1.63 B-parameter GGUF, llama.cpp's weight format) appears in earlier llama-bench logs under a granite-3B size label; the application has since moved to a larger model. Build commands are in `pi/flood/README.md`.

X. CONCLUSION

Seppa separates proposal from judgment: a language model suggests kernels for the Raspberry Pi 5's integrated GPU, and a state-machine MCP server decides what is true about them. On a real flood stencil, this discipline produced a verified 1.58x, proposed in our hand-driven sweep and reproduced within 1% by the machine five of five times. That advantage survives deployment: in the steady-state campaign, 883 steps/s of simulation beside decode at 84% of its solo speed. The implication is broader: commodity edge boards carry more usable silicon than their deployments exercise, and a loop that cannot misreport its own verdicts is a sound way to put that silicon to work.

REFERENCES

- [1] M. Guidolin, A. S. Chen, B. Ghimire, E. C. Keedwell, S. Djordjević, and D. A. Savić. A weighted cellular automata 2D inundation model for rapid flood analysis. *Environmental Modelling & Software* 84:378-394, 2016.
- [2] J. Dibbelt, B. Strasser, and D. Wagner. Customizable Contraction Hierarchies. *ACM Journal of Experimental Algorithmics* 21, Article 1.5, 2016.
- [3] A. Ouyang, S. Guo, et al. KernelBench: Can LLMs Write Efficient GPU Kernels? *arXiv:2502.10517; ICML 2025*.
- [4] X. Li et al. CUDA-L1: Improving CUDA Optimization via Contrastive Reinforcement Learning. *arXiv:2507.14111; ICLR 2026*.
- [5] Burr. Apache Software Foundation (incubating). <https://github.com/apache/burr>
- [6] Model Context Protocol. <https://modelcontextprotocol.io>
- [7] Bonbibib: offline flood-aware routing on a Raspberry Pi 5. <https://github.com/msradam/bonbibib>
- [8] llama.cpp. ggml-org. <https://github.com/ggml-org/llama.cpp>
- [9] V3D driver documentation, Mesa 3D. <https://docs.mesa3d.org/drivers/v3d.html>
- [10] J. Blocklove, S. Garg, R. Karri, and H. Pearce. Chip-Chat: Challenges and Opportunities in Conversational Hardware Design. *ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*, 2023.
- [11] Z. Zhou et al. Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing. *Proc. IEEE* 107(8):1738-1762, 2019.
- [12] S. Boddu and A. Mukherjee. Efficient Edge Deployment of Quantized YOLOv4-Tiny for Aerial Emergency Object Detection on Raspberry Pi 5. *arXiv:2506.09300, 2025*.
- [13] A. Reuther et al. Survey of Machine Learning Accelerators. *IEEE HPEC*, 2020.
- [14] The White House. Building Resilient Supply Chains, Revitalizing American Manufacturing, and Fostering Broad-Based Growth: 100-Day Reviews under Executive Order 14017. June 2021.
- [15] AutoKernel. RightNow AI. <https://github.com/RightNow-AI/autokernel>
- [16] Mesa 25.0.7, `src/broadcom/compiler/vir.c`, `strategies[] compile-fallback table`. <https://gitlab.freedesktop.org/mesa/mesa/-/blob/mesa-25.0.7/src/broadcom/compiler/vir.c>
- [17] C. Baronio, P. Marsella, B. Pan, et al. Kevin: Multi-Turn RL for Generating CUDA Kernels. *arXiv:2507.11948, 2025*.
- [18] J. Woo, S. Zhu, A. Nie, Z. Jia, Y. Wang, and Y. Park. TritonRL: Training LLMs to Think and Code Triton Without Cheating. *arXiv:2510.17891, 2025*.
- [19] D. Sarkar. The Correctness Illusion in LLM-Generated GPU Kernels. *arXiv:2606.20128, 2026*.
- [20] B. Zhao, D. Srikanth, Y. Wu, and Z. Jiang. SpecBench: Measuring Reward Hacking in Long-Horizon Coding Agents. *arXiv:2605.21384, 2026*.
- [21] B. Romera-Paredes, M. Barekatin, A. Novikov, et al. Mathematical discoveries from program search with large language models. *Nature* 625:468-475, 2024.
- [22] A. Novikov, N. Vü, M. Eisenberger, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. Google DeepMind, 2025. *arXiv:2506.13131*.
- [23] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg. VeriGen: A Large Language Model for Verilog Code Generation. *ACM TODAES* 29(3), Article 46, 2024.
- [24] W. L. Oberkamp and C. J. Roy. *Verification and Validation in Scientific Computing*. Cambridge University Press, 2010.